

浙江大学

本科实验报告

课程名称： 电子电路设计实验 II

姓 名：

学 院： 信息与工程学院

专 业： 信息工程

学 号：

指导教师： 施红军、叶险峰、邓靖靖

浙江大学 实验报告

专业： 信息工程

姓名：

学号：

日期： 2026 年 5 月 19 日

地点： 东四-216

课程名称： 电子电路设计实验 II 指导老师： 施红军，叶险峰，邓靖靖 成绩：

实验名称： 多功能数字时钟的设计与制作 实验类型： 设计性实验 同组学生姓名： 卿宸

一、 实验目的

1.1 示例项目回顾——电流电压转换电路设计

- 学习并掌握使用 Altium Designer 软件进行电路原理图绘制与 PCB 布局布线的基本流程。
- 掌握通用模拟运算放大电路（差动放大器、反相加法器）的参数计算与工程调试方法。
- 熟悉元件封装选择、引脚对应关系核查及 PCB 电气规则检查（ERC/DRC）的操作规范。

1.2 多功能数字时钟设计与制作

- 学习掌握用 Arduino UNO 开发板设计并实现多功能数字时钟系统。
- 学习掌握 PCB 电路板的设计和制作，完成 Arduino UNO 扩展板的工程实现。
- 学习掌握 DS1302 实时时钟芯片的通信原理与应用方法。
- 学习掌握 LCD1602 液晶显示屏的驱动与字符显示控制。
- 培养软硬件协同设计能力，积累 Arduino 嵌入式系统开发与联调经验。

二、 实验任务与要求

2.1 I-V 转换电路任务与要求

- 参考给定电路原理图，设计两级级联运放电路，将 4~20 mA 标准电流信号高精度线性转换为 0~10 V 标准电压信号。
- 在 Altium Designer 软件中完成元器件选型、封装匹配及 PCB 布局布线，板尺寸要求宽不超过 99 mm、高不超过 49 mm。

2.2 多功能数字时钟任务与要求

以 Arduino UNO 为主控，独立设计并制作扩展板，需完整实现以下核心功能：

- 实时显示当前时间（时:分:秒）、日期（年-月-日）及星期信息。
- 具备掉电保护机制，通过 DS1302 芯片在断电后持续保存时间数据。
- 支持通过板载独立按键对时间与日期进行人工校准与设置。

4. 整点报时：每逢整点驱动蜂鸣器鸣响 500 ms。
5. 自定义闹钟：允许用户设定闹钟时间，到时持续鸣响 5 s。
6. 实时采集并动态显示当前环境温度（通过 LM35 模拟传感器）。
7. 支持用户自定义超温报警阈值，超温时驱动蜂鸣器报警 500 ms。
8. 按键复合控制逻辑：单按“选择键”进入时间设置，同按“+”与“-”键进入闹钟/报警温度配置界面；在设置模式下再按“选择键”可切换光标至不同参数位，由“+”/“-”键修改。
9. 编制、编译并调试全部控制程序，完成软硬件综合联调与功能验证。

三、 实验原理

3.1 I-V 转换电路原理

在工业自动化控制中，变送器输出的 4~20 mA 标准电流信号需转换为 0~10 V 电压信号以供后级处理。本实验采用两级运放级联实现该转换。

第一级：差动放大器。电阻 $R_1 = 250\ \Omega$ 跨接于电流源两端，对输入电流 I_i 进行采样，产生压降。运放 A_1 与四个匹配电阻（ $R_2 = R_3 = R_4 = R_5 = 10\ \text{k}\Omega$ ）构成差动放大器，输出电压为：

$$V_{o1} = -I_i R_1$$

当 I_i 在 4~20 mA 变化时， V_{o1} 在 $-1 \sim -5\ \text{V}$ 范围内线性变化。

第二级：反相加法器。运放 A_2 构成反相加法器电路，其输出表达式为：

$$V_o = -\frac{R_f}{R_6} V_{o1} - \frac{R_f}{R_7} V_f = \frac{R_f}{R_6} I_i R_1 - \frac{R_f}{R_7} V_f$$

通过合理选取 $R_6 = R_7 = 10\ \text{k}\Omega$ ，令 $R_f = 25\ \text{k}\Omega$ （由 $20\ \text{k}\Omega$ 固定电阻串联 $10\ \text{k}\Omega$ 电位器 R_{w1} 实现微调），并利用 5.1 V 稳压管分压网络产生基准电压 $V_f = 1\ \text{V}$ ，可使 I_i 从 4~20 mA 变化时， V_o 对应从 0~10 V 线性变化。

稳压管限流电阻：

$$R_{11} = \frac{12\ \text{V} - 5.1\ \text{V}}{10\ \text{mA} + 5\ \text{mA}} \approx 470\ \Omega$$

下端分压网络由 $R_{12} = 510\ \Omega$ 与电位器 $R_{w2} = 500\ \Omega$ 串联，基准电压 V_f 可在 0~2.5 V 间调节。

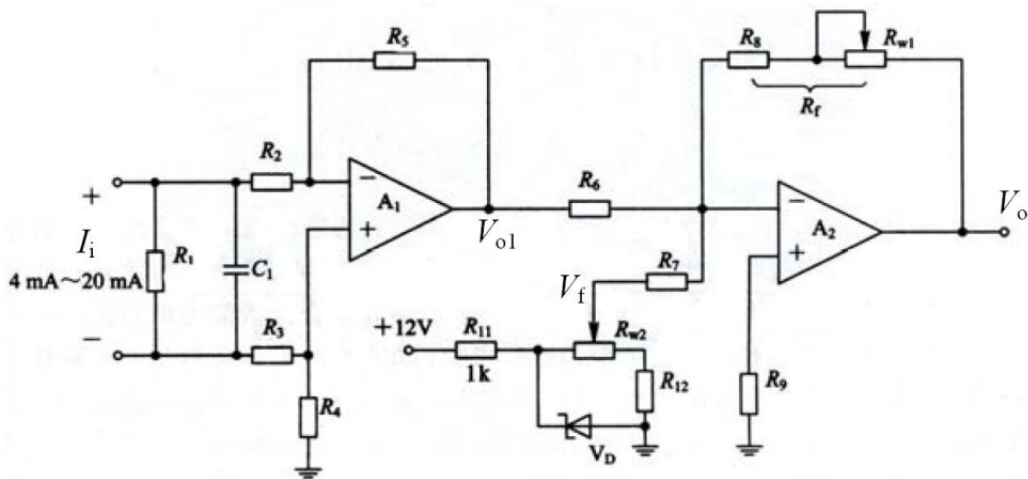


图 3.1: I-V 转换电路图

3.2 多功能数字时钟系统原理

系统以 Arduino UNO 为控制核心，通过串行与并行总线协同各外设模块工作。整体硬件框图如图 3.2 所示。

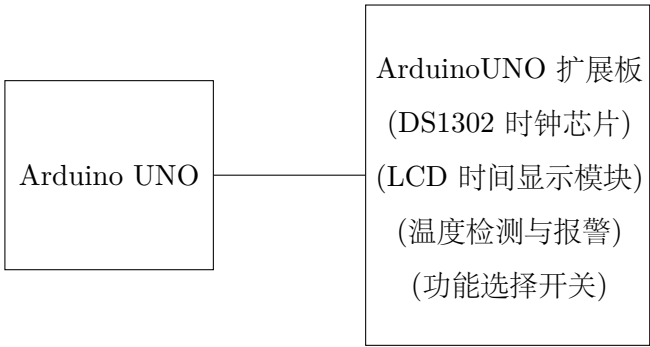


图 3.2: 多功能数字时钟硬件框图

3.2.1 DS1302 实时时钟芯片

DS1302 是一款高性能、低功耗实时时钟芯片，内置 31 字节静态 RAM，采用 SPI 三线同步串行接口（RST/CLK/DAT）与主控通信。芯片支持秒、分、时、日、星期、月、年计数，具备自动月长调整与闰年补偿功能。工作电压宽达 2.5~5.5 V，采用主备双电源供电，支持涓流充电，可在主电源断电后由备用纽扣电池持续维持计时。

本实验采用 DS1302 模块（五引脚排针封装：VCC、GND、CLK、DAT、RST），模块上已集成晶振与备用电池，直接通过排针与扩展板连接，无需在 PCB 上单独设计晶振与备用电源电路。

3.2.2 LCD1602 液晶显示模块

LCD1602 是 16 列 ×2 行的字符型液晶显示器，内部集成驱动电路，兼容 HD44780 控制器标准。本实验采用 4 线数据模式（DB4~DB7）驱动，配合 RS（寄存器选择）和 E（使能）信号实现字符写

入。通过串联电位器调节 V0 引脚可控制背光对比度。

3.2.3 LM35 温度传感器

LM35 为高精度模拟温度传感器, 输出电压与摄氏温度成线性正比关系, 灵敏度为 $10\text{ mV}/^{\circ}\text{C}$, 无需外部校准。测量范围 -55°C 至 $+150^{\circ}\text{C}$, 工作电流约 60 A , 供电电压 $4\sim 30\text{ V}$ 。Arduino 的模拟引脚 A3 读取其输出电压, 按下式换算为摄氏温度:

$$T(^{\circ}\text{C}) = \frac{500 \times a}{1023}$$

其中 a 为 ADC 采样值 (10 位, 参考电压 5 V)。

3.2.4 蜂鸣器报警

有源蜂鸣器由 Arduino 数字引脚 D13 直接驱动, 通过 `tone()` 和 `noTone()` 函数控制鸣响时长, 分别用于整点报时 (500 ms)、闹钟触发 (5000 ms) 和超温报警 (500 ms)。

四、 实验内容

4.1 示例项目: I-V 转换电路设计

4.1.1 电路原理图绘制

在 Altium Designer 中按照以下步骤完成原理图:

1. 根据计算结果确定各元件参数 ($R_1 = 250\Omega$, $R_2 \sim R_5 = 10\text{ k}\Omega$, $R_6 = R_7 = 10\text{ k}\Omega$, $R_8 = 20\text{ k}\Omega$, R_f 由 R_{w1} 微调, $R_{11} = 470\Omega$, $R_{12} = 510\Omega$, $R_{w2} = 500\Omega$, 补偿电阻 $R_9 \approx 4.3\text{ k}\Omega$, 运放选用 OP07, 电容 $C_1 = 100\text{ pF}$)。
2. 为每个元件指定正确的封装形式: OP07 选 DIP8, $1/4\text{ W}$ 电阻选 AXIAL0.4, 瓷片电容选 RAD0.2, 稳压二极管选 DIODE0.4, 电位器选 VR5。
3. 注意电位器引脚对应关系: VR5 封装中滑动活动端 (中间引脚) 的 PCB 引脚编号与原理图符号默认分配不一致, 需手动修改引脚网络标号, 确保原理图与实物一致。
4. 添加电源接口 (5 引脚 Header5, 封装 PowerPlug5)、关键测试点 Vo1、Vf、GND (均采用 SIP2 排针) 及输入接口 (SIP4 排针)。
5. 完成电气规则检查 (ERC), 确认无错误后生成网表。

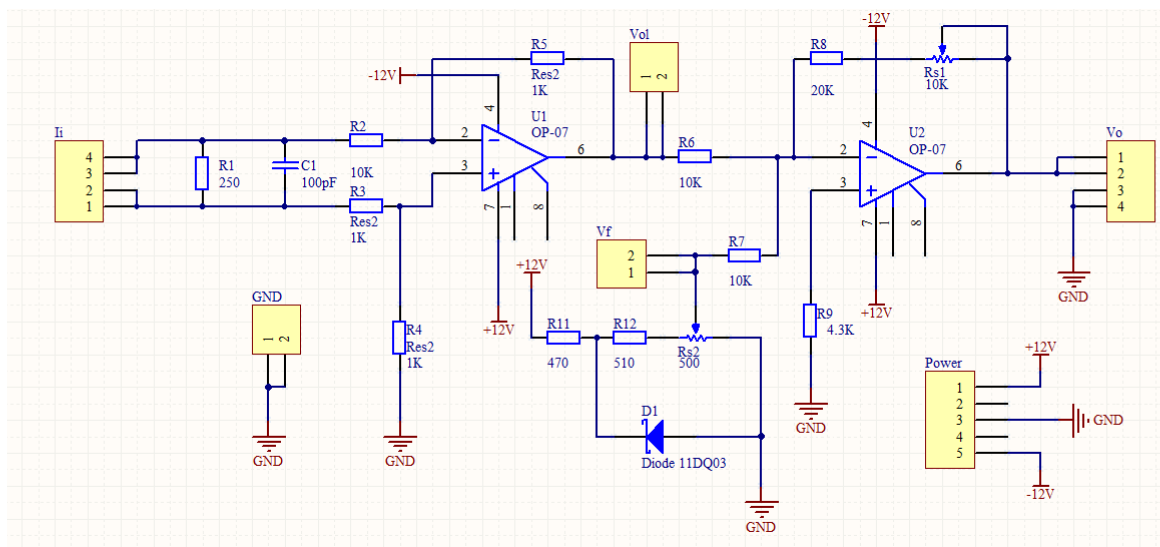


图 4.1: I-V 转换电路原理图

4.1.2 PCB 版图绘制

1. 在 keepout 层绘制板框，确保板宽不超过 99 mm、板高不超过 49 mm。
2. 按布局三原则放置元件：同一基本单元电路集中放置；各单元按信号流向左右摆放（差动放大 → 反相加法 → 稳压分压）；单元内部元件位置参照原理图相对位置。
3. 手工布线：信号线线宽 20 mil，电源线及地线线宽 50 mil；走线拐角采用 45° 斜角，禁止直角走线；合理利用双层（Top/Bottom）布线及过孔（Via）完成复杂交叉连接。
4. 在丝印层（Top Overlay）添加电路名称、作者标识（格式：班次 _ 桌号 _ 学号后四位）、电源极性标注（V+/V-/GND）等信息。
5. 在 PCB 四角放置内外径均为 125 mil 的安装孔（Pad），完成板边设计。

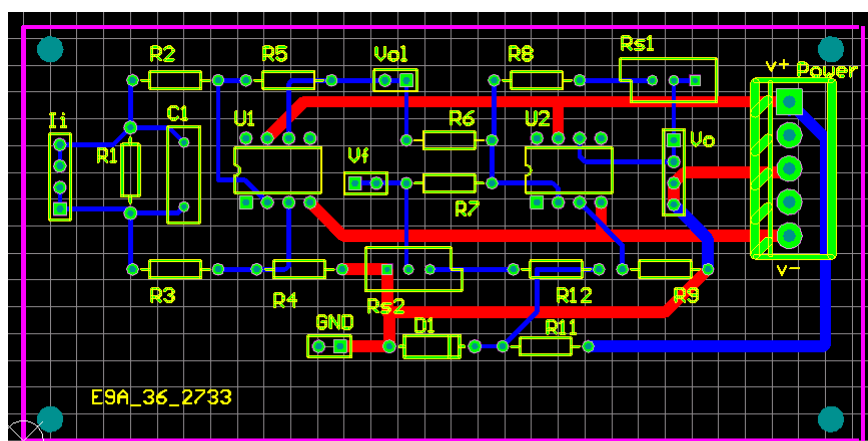


图 4.2: I-V 转换电路 PCB 版图

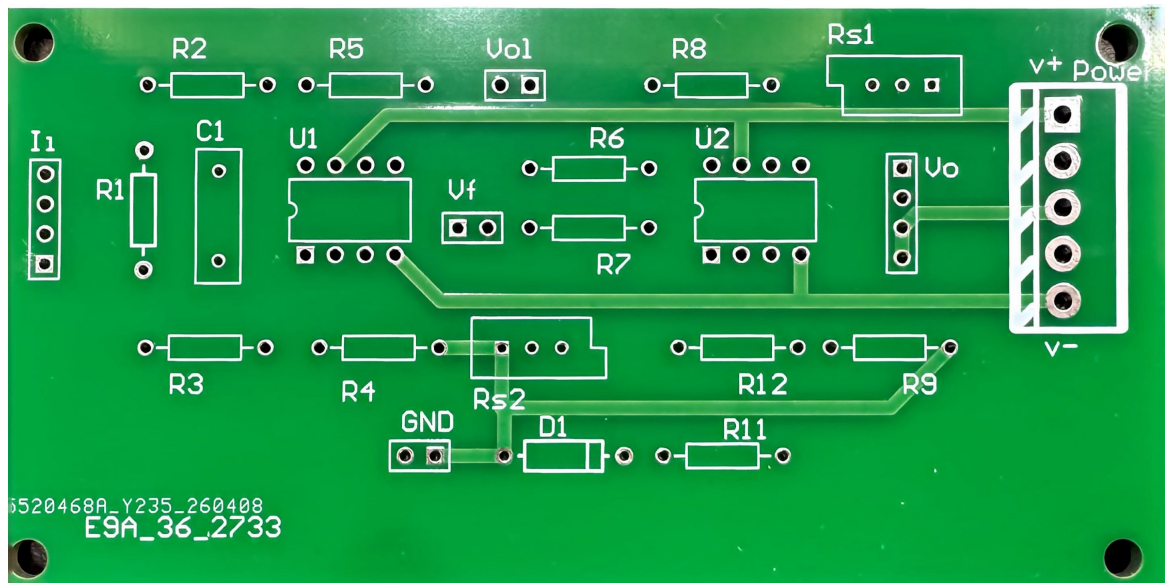


图 4.3: I-V 转换电路 PCB 板实物

4.2 数字时钟项目：硬件设计

4.2.1 引脚连接关系

Arduino UNO 与各外设模块的引脚映射关系如表 4.1 所示。

表 4.1: Arduino UNO 与各外设模块引脚映射表

外设模块	模块引脚	Arduino 引脚
DS1302 时钟芯片	RST	D8
DS1302 时钟芯片	DAT (I/O)	D9
DS1302 时钟芯片	CLK	D10
LCD1602 液晶屏	RS	D12
LCD1602 液晶屏	E (Enable)	D11
LCD1602 液晶屏	DB4	D5
LCD1602 液晶屏	DB5	D4
LCD1602 液晶屏	DB6	D3
LCD1602 液晶屏	DB7	D2
按键开关 S1 (选择)	—	A0
按键开关 S2 (加)	—	A1
按键开关 S3 (减)	—	A2
LM35 温度传感器	Vo	A3
蜂鸣器	—	D13

4.2.2 电路原理图绘制

在 Altium Designer 中新建时钟扩展板工程，按引脚映射表完成各模块与 Arduino UNO 封装之间的电气连接。主要注意事项：

- Arduino UNO 封装必须使用实验室指定的专用库文件（Lixh 库），否则引脚孔距将与实物不符。
- DS1302 采用五引脚排针封装（Header5），在原理图中手动设置 RST/DAT/CLK/GND/VCC 五个引脚的网络标号，与排针物理引脚一一对应。
- LCD1602 的 R/W 引脚接地（固定为写模式），VSS 接地，VCC 接 5 V；V0 引脚串联电位器接地以调节对比度；背光 BLA 接 5 V，BLK 接地。
- 电位器中间活动端引脚的网络标号须手动核查修正，避免与封装引脚错位。

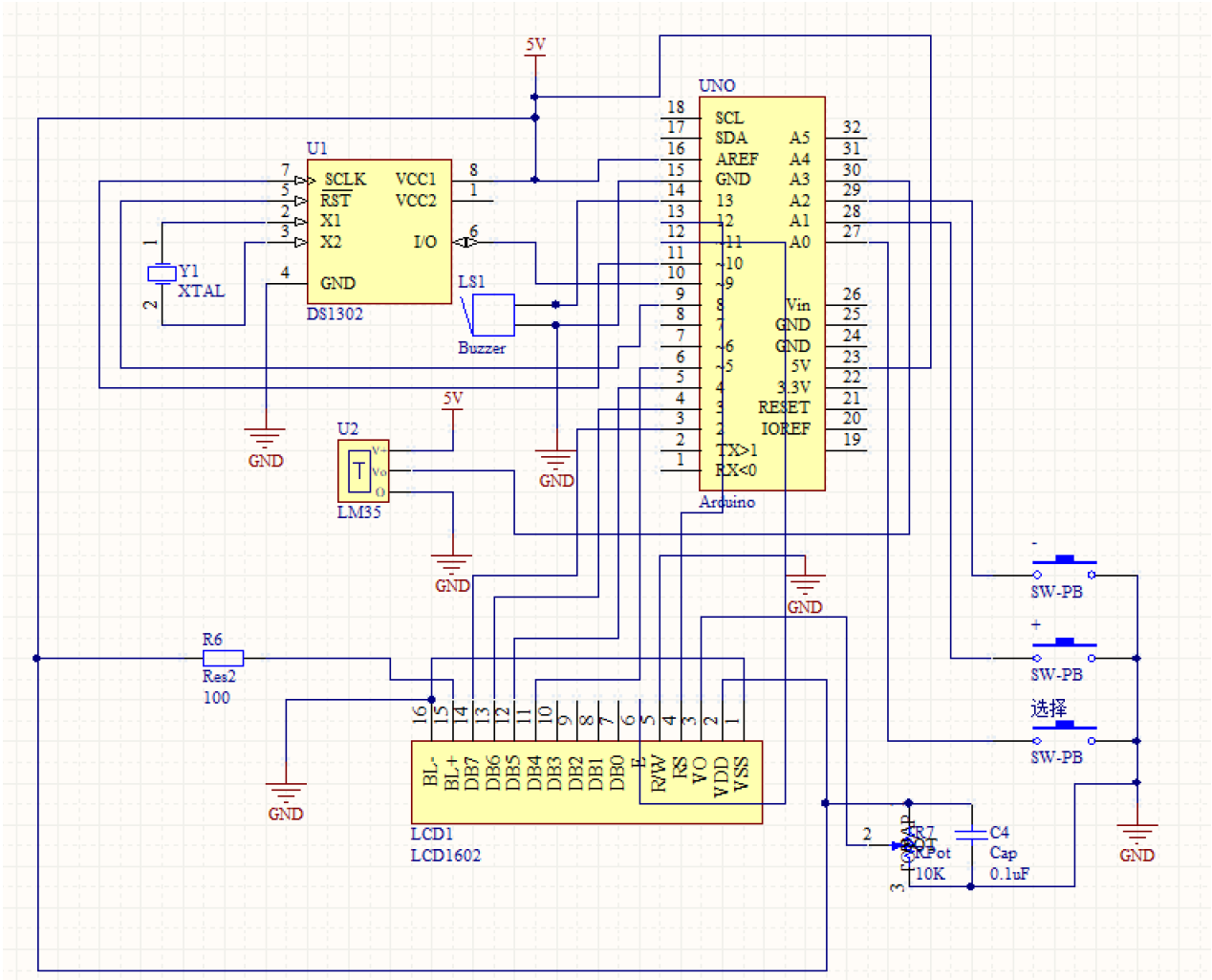


图 4.4: 多功能数字时钟电路原理图

4.2.3 PCB 扩展板版图绘制

1. 在 keepout 层绘制板框，物理尺寸要求在 99×99 mm 以内。
2. 由于扩展板直插叠于 Arduino 核心板之上，布局时可将 DS1302 模块、LM35、蜂鸣器及按键等元件布置在 Arduino 核心板投影区域上方，合理利用立体空间以减少外部连线长度。
3. 走线约束：信号线 20 mil，电源线与地线 50 mil，禁止直角走线，灵活运用双层布线及过孔完成复杂走线。
4. 在丝印层放置项目标识内容，完成整板设计。

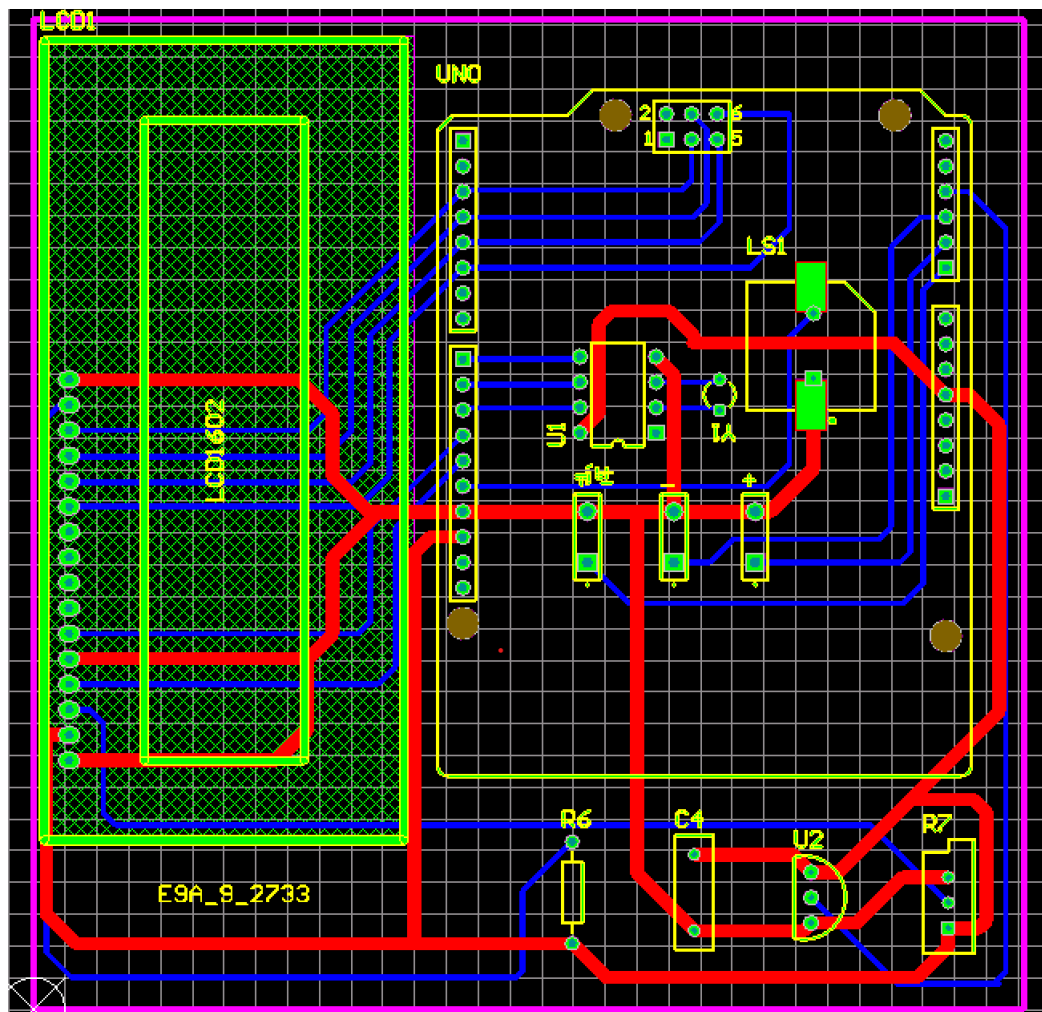


图 4.5: 多功能数字时钟 PCB 扩展板版图

4.3 数字时钟项目：软件设计

程序采用模块化函数架构，主循环（`loop()`）以高频轮询模式依次调用各功能子模块，整体逻辑如下：

1. 时间显示模块（`Display()`）：依次调用 `time()`、`Day()`、`Month()`、`Year()`、`Week()` 五个子函数，按秒 → 分 → 时 → 日 → 月 → 年 → 星期的进位顺序更新并在 LCD 上格式化显示全部时间信息。
2. 时间设置模块（`Set_Clock()`）：检测“选择键”（A0）是否按下，若按下则进入死循环阻塞设置模式。每次再按选择键，`chose` 变量递增，光标依次定位到时 → 分 → 秒 → 日 → 月 → 年各位；“+”键（A1）与“-”键（A2）对应位置加减，采用软件延时消抖（`ButtonDelay = 10 ms`）防止误触。
3. 闹钟/温度报警设置模块（`Set_Alarm()`）：同时按下“+”与“-”键触发，进入闹钟及报警温度综合设置界面，通过 `alarm_choose` 变量切换设置项（闹钟时 → 闹钟分 → 报警温度），按选择键第四次自动退出。

4. 整点报时 (Point_Time_Alarm()): 当 minute == 0 && second == 0 时触发蜂鸣器鸣响 500 ms。
5. 闹钟驱动 (Clock_Alarm()): 当当前时间与设定闹钟时间（时、分、秒）完全匹配时，触发蜂鸣器持续鸣响 5000 ms。
6. 温度采集与报警 (GetTemperatures()、Temperatures_Alarm()): 读取 A3 引脚模拟量，换算为摄氏温度后显示；若超过设定阈值则触发报警。
7. DS1302 时间同步：每次 loop() 末尾将 Arduino 内存中的当前时间写回 DS1302 芯片，保证断电后芯片能从最新时间继续计时。

五、 实验结果与分析

电路焊接完成后，接入 ±12 V 电源，通过信号发生器模拟 4~20 mA 电流输入，调试步骤如下：

1. 零点调零：输入电流设为 4 mA，调节电位器 R_{w2} 改变基准电压 V_f ，使第二级输出 $V_o = 0\text{ V}$ （此时 $V_{o1} = -1\text{ V}$ ， $V_f = 1\text{ V}$ ）。
2. 满量程校准：输入电流设为 20 mA，调节电位器 R_{w1} 改变 R_f ，使输出 $V_o = 10\text{ V}$ 。
3. 线性度验证：在 4~20 mA 范围内取若干点，测量对应输出电压，验证线性关系。

表 5.1: I-V 转换电路测量数据（占位符，请填入实测值）

输入电流 I_i (mA)	理论输出 V_o (V)	实测输出 V_o (V)	误差 (mV)
4	0	0	0
8	2.5	2.51	0.01
12	5	5.02	0.02
16	7.5	7.53	0.03
20	10	10.03	0.03

调试结果表明，通过两步调节（零点调零与满量程校准）可实现精确的 4~20 mA 到 0~10 V 线性转换，电路工作稳定。

六、 实验结论

1. 通过示例项目，系统掌握了 Altium Designer 进行原理图设计与 PCB 版图绘制的完整流程，理解了差动放大器与反相加法器的参数计算方法，能够独立实现 4~20 mA 到 0~10 V 的精确线性电流电压转换。

2. 完成了多功能数字时钟 Arduino 扩展板的设计与制作, 掌握了 DS1302 时钟芯片 (SPI 三线串行通信)、LCD1602 (4 线并行驱动) 及 LM35 (模拟温度采样) 的使用方法, 实现了题目要求的全部九项功能。
3. 软件设计方面, 通过模块化函数架构与主循环轮询调度, 合理组织了时间显示、按键状态机、闹钟驱动和环境监测等功能模块, 验证了软件延时消抖对按键误触的有效抑制。
4. 工程实践中积累了重要经验: 元件封装选择需严格核查实物尺寸; 电位器等元件的原理图引脚与封装引脚的对应关系须手动修正; DS1302 应优先选用带晶振和备用电池的模块封装而非裸芯片, 以保证计时精度和掉电保存功能的可靠实现。
5. 不同于纯硬件项目, Arduino 单片机项目将硬件设计能力与嵌入式编程思维深度融合, 深刻体会到充分理解器件功能再进行设计的重要性——设计阶段的疏漏往往会在调试阶段带来成倍的困难。

七、 源代码与分析

以下为多功能数字时钟 Arduino 完整控制程序, 关键函数已添加注释说明。

Listing 1: 多功能数字时钟 Arduino 完整程序

```
1  /* 引脚连接说明
2   * LCD RS   -> D12,  LCD Enable -> D11
3   * LCD D4   -> D5,   LCD D5      -> D4
4   * LCD D6   -> D3,   LCD D7      -> D2
5   * DS1302 RST -> D8, DAT -> D9, CLK -> D10
6   * 选择键 -> A0, 加键 -> A1, 减键 -> A2
7   * LM35     -> A3,   蜂鸣器 -> D13
8   */
9  #include <DS1302.h>
10 #include <LiquidCrystal.h>
11
12 // 初始化 LCD (RS, E, D4, D5, D6, D7)
13 LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
14
15 #define choose A0  // 选择按键端口
16 #define add    A1  // 加按键端口
17 #define minus  A2  // 减按键端口
18 #define Tone   13  // 蜂鸣器端口
19
20 uint8_t CE_PIN = 8;  // DS1302 RST
21 uint8_t IO_PIN  = 9;  // DS1302 DAT
```

```
22  uint8_t SCLK_PIN = 10;  // DS1302 CLK
23  DS1302 rtc(CE_PIN, IO_PIN, SCLK_PIN);
24
25  unsigned long seconds;
26  // 时间进位缓冲变量
27  int s = 0, m = 0, h = 0, d = 0, mon = 0, y = 0;
28  // 当前显示时间
29  int second = 0, minute = 0, hour = 0,
30      day = 0, month = 0, year = 0;
31  // 初始基准时间 (从 DS1302 读取后写入)
32  int SECOND = 0, MINUTE = 0, HOUR = 0,
33      DAY = 0, MONTH = 0, YEAR = 0;
34
35  int chose = 0, alarm_choose = 0;
36  int ButtonDelay = 10;  // 消抖延时 (ms)
37  int frequency = 2093;  // 蜂鸣器频率 (Hz)
38  int alarm_hour = 7, alarm_minute = 30, alarm_second = 0;
39  float Temperatures, Temp_Alarm = 30;
40
41  /* 格式化输出: 在 LCD 指定列行位置显示两位数 (不足两位前补零) */
42  void FormatDisplay(int col, int row, int num) {
43      lcd.setCursor(col, row);
44      if (num < 10) lcd.print("0");
45      lcd.print(num);
46  }
47
48  /* 时间计算: 利用 millis() 计算秒数, 实现秒->分->时进位 */
49  void time() {
50      second = (SECOND + seconds) % 60;
51      m      = (SECOND + seconds) / 60;
52      FormatDisplay(6, 1, second);
53
54      minute = (MINUTE + m) % 60;
55      h      = (MINUTE + m) / 60;
56      FormatDisplay(3, 1, minute);
57
58      hour = (HOUR + h) % 24;
59      d    = (HOUR + h) / 24;
60      FormatDisplay(0, 1, hour);
```

```
61
62     lcd.setCursor(2, 1); lcd.print(":");
63     lcd.setCursor(5, 1); lcd.print(":");
64 }
65
66 /* 计算指定年月的天数 (含闰年判断) */
67 int Days(int year, int month) {
68     int days = 0;
69     if (month != 2) {
70         switch (month) {
71             case 1: case 3: case 5: case 7:
72             case 8: case 10: case 12: days = 31; break;
73             case 4: case 6: case 9: case 11: days = 30; break;
74         }
75     } else {
76         // 闰年: 能被4整除但不能被100整除, 或能被400整除
77         if (year % 4 == 0 && year % 100 != 0
78             || year % 400 == 0) days = 29;
79         else days = 28;
80     }
81     return days;
82 }
83
84 /* 日期进位处理: 处理月末/月初跨越及月份进位 */
85 void Day() {
86     int days = Days(year, month);
87     int days_up = (month == 1) ?
88         Days(year - 1, 12) : Days(year, month - 1);
89
90     day = (DAY + d) % days;
91     if (day == 0) day = days;
92     if ((DAY + d) == days + 1) { DAY -= days; mon++; }
93     if ((DAY + d) == 0) { DAY += days_up; mon--; }
94     FormatDisplay(8, 0, day);
95 }
96
97 /* 月份进位处理 */
98 void Month() {
99     month = (MONTH + mon) % 12;
```

```
100     if (month == 0) month = 12;
101     y = (MONTH + mon - 1) / 12;
102     FormatDisplay(5, 0, month);
103     lcd.setCursor(7, 0); lcd.print('-');
104 }
105
106 /* 年份显示 (自动四位补零) */
107 void Year() {
108     year = (YEAR + y) % 9999;
109     if (year == 0) year = 9999;
110     lcd.setCursor(0, 0);
111     if (year < 1000) lcd.print("0");
112     if (year < 100)  lcd.print("0");
113     if (year < 10)   lcd.print("0");
114     lcd.print(year);
115     lcd.setCursor(4, 0); lcd.print('-');
116 }
117
118 /* 蔡勒公式推算星期并输出英文缩写 */
119 void Week(int y, int m, int d) {
120     if (m == 1) m = 13;
121     if (m == 2) m = 14;
122     int week = (d + 2*m + 3*(m+1)/5 + y
123               + y/4 - y/100 + y/400) % 7 + 1;
124     String weekstr = "";
125     switch (week) {
126         case 1: weekstr = "Mon. "; break;
127         case 2: weekstr = "Tues. "; break;
128         case 3: weekstr = "Wed. "; break;
129         case 4: weekstr = "Thur. "; break;
130         case 5: weekstr = "Fri. "; break;
131         case 6: weekstr = "Sat. "; break;
132         case 7: weekstr = "Sun. "; break;
133     }
134     lcd.setCursor(11, 0); lcd.print(weekstr);
135 }
136
137 /* 集成显示函数: 按进位顺序依次更新全部时间信息 */
138 void Display() {
```

```
139     time();
140     Day(); Month(); Year();
141     Week(year, month, day);
142 }
143
144 /* 光标闪烁显示 (100ms 可见 + 100ms 不可见) */
145 void DisplayCursor(int rol, int row) {
146     lcd.setCursor(rol, row);
147     lcd.cursor();    delay(100);
148     lcd.noCursor(); delay(100);
149 }
150
151 /* 设置初始基准时间 */
152 void set(int y, int mon, int d, int h, int m, int s) {
153     YEAR = y; MONTH = mon; DAY = d;
154     HOUR = h; MINUTE = m; SECOND = s;
155 }
156
157 /* 通过加减键调节指定时间位 (引用传参) */
158 void Set_Time(int rol, int row, int &Time) {
159     DisplayCursor(rol, row);
160     if (digitalRead(add) == LOW) {
161         delay(ButtonDelay);
162         if (digitalRead(add) == LOW) Time++;
163         Display();
164     }
165     if (digitalRead(minus) == LOW) {
166         delay(ButtonDelay);
167         if (digitalRead(minus) == LOW) Time--;
168         Display();
169     }
170 }
171
172 /* 时间设置主状态机: chose 递增依次切换光标至各时间位 */
173 void Set_Clock() {
174     if (digitalRead(choose) == LOW) {
175         lcd.setCursor(9, 1); lcd.print("SetTime");
176         while (1) {
177             if (digitalRead(choose) == LOW) {
```



```
178         delay(ButtonDelay);
179         if (digitalRead(choose) == LOW) chose++;
180     }
181     seconds = millis() / 1000;
182     Display();
183     if (chose == 1) Set_Time(1, 1, HOUR);
184     else if (chose == 2) Set_Time(4, 1, MINUTE);
185     else if (chose == 3) Set_Time(7, 1, SECOND);
186     else if (chose == 4) Set_Time(9, 0, DAY);
187     else if (chose == 5) Set_Time(6, 0, MONTH);
188     else if (chose == 6) Set_Time(3, 0, YEAR);
189     else if (chose >= 7) { chose = 0; break; }
190 }
191 }
192 }
193
194 /* 设置闹钟小时 (0-23 循环) */
195 void Set_Alarm_Hour() {
196     DisplayCursor(1, 1);
197     if (digitalRead(add) == LOW) {
198         delay(ButtonDelay);
199         if (digitalRead(add) == LOW) {
200             alarm_hour++;
201             if (alarm_hour == 24) alarm_hour = 0;
202             FormatDisplay(0, 1, alarm_hour);
203         }
204     }
205     if (digitalRead(minus) == LOW) {
206         delay(ButtonDelay);
207         if (digitalRead(minus) == LOW) {
208             alarm_hour--;
209             if (alarm_hour == -1) alarm_hour = 23;
210             FormatDisplay(0, 1, alarm_hour);
211         }
212     }
213 }
214
215 /* 设置闹钟分钟 (0-59 循环) */
216 void Set_Alarm_Minute() {
```

```
217     DisplayCursor(4, 1);
218     if (digitalRead(add) == LOW) {
219         delay(ButtonDelay);
220         if (digitalRead(add) == LOW) {
221             alarm_minute++;
222             if (alarm_minute == 60) alarm_minute = 0;
223             FormatDisplay(3, 1, alarm_minute);
224         }
225     }
226     if (digitalRead(minus) == LOW) {
227         delay(ButtonDelay);
228         if (digitalRead(minus) == LOW) {
229             alarm_minute--;
230             if (alarm_minute == -1) alarm_minute = 59;
231             FormatDisplay(3, 1, alarm_minute);
232         }
233     }
234 }
235
236 /* 设置报警温度阈值 */
237 void Set_Alarm_Temp() {
238     DisplayCursor(10, 1);
239     if (digitalRead(add) == LOW) {
240         delay(ButtonDelay);
241         if (digitalRead(add) == LOW) Temp_Alarm++;
242     }
243     if (digitalRead(minus) == LOW) {
244         delay(ButtonDelay);
245         if (digitalRead(minus) == LOW) Temp_Alarm--;
246     }
247 }
248
249 /* 进入报警综合设置 (同按 add+minus 触发) */
250 void Set_Alarm() {
251     if (digitalRead(add) == LOW && digitalRead(minus) == LOW) {
252         alarm_hour = hour; alarm_minute = minute;
253         lcd.setCursor(0, 0); lcd.print("set alarm      ");
254         lcd.setCursor(6, 1); lcd.print("00");
255         while (1) {
```

```
256         if (digitalRead(choose) == LOW) {
257             delay(ButtonDelay);
258             if (digitalRead(choose) == LOW) alarm_choose++;
259         }
260         lcd.setCursor(9, 1); lcd.print(Temp_Alarm);
261         lcd.setCursor(14, 1); lcd.print((char)223); // 度号
262         lcd.setCursor(15, 1); lcd.print("C");
263         if (alarm_choose == 1) Set_Alarm_Hour();
264         else if (alarm_choose == 2) Set_Alarm_Minute();
265         else if (alarm_choose == 3) Set_Alarm_Temp();
266         else if (alarm_choose >= 4) { alarm_choose = 0; break; }
267     }
268 }
269 }
270
271 /* 整点报时: 分=0 且秒=0 时鸣响 500 ms */
272 void Point_Time_Alarm() {
273     if (minute == 0 && second == 0) {
274         tone(Tone, frequency); delay(500); noTone(Tone);
275     }
276 }
277
278 /* 闹钟驱动: 时分秒全匹配时鸣响 5 s */
279 void Clock_Alarm() {
280     if (hour == alarm_hour && minute == alarm_minute
281         && second == alarm_second) {
282         tone(Tone, frequency); delay(5000); noTone(Tone);
283     }
284 }
285
286 /* 读取 LM35 温度并显示 (转换公式: T = 500*ADC/1023) */
287 void GetTemperatures() {
288     long a = analogRead(A3);
289     Temperatures = 500.0 * a / 1023.0;
290     lcd.setCursor(9, 1); lcd.print(Temperatures);
291     lcd.setCursor(14, 1); lcd.print((char)223);
292     lcd.setCursor(15, 1); lcd.print("C");
293 }
294
```

```
295  /* 超温报警 */
296  void Temperatures_Alarm() {
297      if (Temperatures >= Temp_Alarm) {
298          tone(Tone, frequency); delay(500); noTone(Tone);
299      }
300  }
301
302  /* 初始化: 引脚模式、LCD 和 DS1302 配置 */
303  void setup() {
304      for (int i = 2; i <= 13; i++) pinMode(i, OUTPUT);
305      digitalWrite(add, HIGH);
306      digitalWrite(minus, HIGH);
307      digitalWrite(choose, HIGH);
308      lcd.begin(16, 2);
309      // 解除 DS1302 写保护, 从芯片读取当前时间
310      rtc.writeProtect(false);
311      rtc.halt(false);
312      Time t = rtc.getTime();
313      set(t.year, t.mon, t.date, t.hour, t.min, t.sec);
314  }
315
316  /* 主循环: 轮询调度各功能模块 */
317  void loop() {
318      seconds = millis() / 1000;
319      Display();           // 1. 刷新时间显示
320      Set_Clock();         // 2. 检测时间设置按键
321      Set_Alarm();         // 3. 检测闹钟设置按键
322      Point_Time_Alarm();  // 4. 整点报时检查
323      Clock_Alarm();       // 5. 闹钟触发检查
324      GetTemperatures();   // 6. 读取并显示温度
325      Temperatures_Alarm(); // 7. 超温报警检查
326      // 将内存中的当前时间同步写回 DS1302 芯片
327      Time t = rtc.getTime();
328      rtc.setTime(t.hour, t.min, t.sec);
329      rtc.setDate(t.date, t.mon, t.year);
330      set(t.year, t.mon, t.date, t.hour, t.min, t.sec);
331  }
```

八、 讨论与心得

本次实验综合运用了硬件电路设计、PCB 版图绘制与 Arduino 嵌入式编程三方面的知识。

在原理图设计阶段，最大的挑战在于保证元件库中的封装引脚与实际器件物理引脚的一一对应。以电位器为例，标准库中电位器的滑动活动端默认对应 3 号引脚，而实验室实际使用的 3296 型电位器，其正中间的活动端物理引脚编号为 2，若在原理图阶段忽略手动修正，则 PCB 阶段极可能产生错误的连接关系，给后续调试带来极大困难。这一细节在实验过程中付出了一定代价，由此深刻认识到“先充分了解器件特性，再进行电路图绘制”的重要性。

在 DS1302 的封装选择上，起初误以为直接使用裸八引脚芯片封装与使用五引脚排针封装是等价的。实际接触电路板后才意识到：DS1302 模块不仅提供引脚引出，还集成了晶振和备用纽扣电池——晶振为芯片提供基准频率以保证计时精度（外接晶振在低功耗模式下仅消耗 0.5~2 A），备用电池在主电源断电后维持芯片持续计时。若直接使用裸芯片而不接晶振，DS1302 的计时功能将完全失效。这一教训说明，电路设计前必须充分阅读 Datasheet，理解器件的完整工作条件。

在 PCB 布线阶段，数字时钟项目因引入了 Arduino UNO 这一较大的封装，走线复杂度显著高于示例项目，需要灵活运用双层布线及过孔才能在有限空间内完成整洁的布线。遵循“同类元件聚集、按信号流向摆放”的布局原则，对减少布线交叉、提高最终 PCB 质量起到了关键作用。

软件程序方面，本次实验是目前接触到复杂度最高的 Arduino 工程，通过定义功能子函数并在主循环中轮询调用的模块化架构，有效降低了程序的理解与维护难度，也为后续功能扩展预留了良好的接口。